

Day 5 Learning

#Java Collection Framework:

The Collection Framework is a unified architecture for storing and manipulating a group of objects. It is located in the `java.util` package.

1. The Iterable Interface:

This is the root interface of the entire collection hierarchy (except Map). It allows an object to be the target of the "for-each loop" statement.

2. The Collection Interface:

It extends `Iterable` and is the interface which most collections implement. It declares the core methods like `add()`, `remove()`, `size()`, and `clear()`.

3. The List Interface:

An ordered collection (also known as a sequence). It can contain duplicate elements.

A. ArrayList (Most Popular):

- **Implementation:** Uses a dynamic array to store elements.
- **Performance:** Fast for retrieving data ($O(1)$) but slow for insertion/deletion in the middle ($O(n)$).
- **Constructors:**
 - `ArrayList()`: Creates an empty list with initial capacity 10.
 - `ArrayList(int initialCapacity)`: Sets a specific size.
 - `ArrayList(Collection<? extends E> c)`: Creates a list from another collection.

B. LinkedList:

- **Implementation:** Doubly linked list.
- **Performance:** Fast for insertion/deletion ($O(1)$) but slow for search ($O(n)$).
- **Constructors:**
 - `LinkedList()`: Empty list.
 - `LinkedList(Collection<? extends E> c)`: From another collection.

4. The Set Interface:

A collection that **cannot contain duplicate elements**. It models the mathematical set abstraction.

A. HashSet:

- **Implementation:** Backed by a HashMap.
- **Ordering:** No guaranteed order.
- **Constructors:**
 - **HashSet():** Default capacity 16, load factor 0.75.
 - **HashSet(int initialCapacity):** Custom capacity.

B. LinkedHashSet

- **Implementation:** Hash table and linked list.
- **Ordering:** Maintains **insertion order**.

C. TreeSet

- **Implementation:** NavigableSet backed by a TreeMap (Red-Black tree).
- **Ordering:** Elements are stored in **sorted order** (Natural or Comparator).

5. The Queue & Deque Interface

Designed for holding elements prior to processing. (First-In-First-Out).

A. PriorityQueue

- **Behavior:** Elements are ordered according to their natural ordering or a custom comparator.
- **Constructors:** **PriorityQueue(int initialCapacity, Comparator<? super E> comparator).**

B. ArrayDeque

- **Implementation:** Resizable array implementation of the Deque interface.
- **Use Case:** Faster than Stack when used as a stack, and faster than LinkedList when used as a queue.

6. The Map Interface (Separate Hierarchy)

An object that maps keys to values. A map **cannot contain duplicate keys**.

A. HashMap

- **Implementation:** Hash table based.
- **Nulls:** Allows one null key and multiple null values.
- **Performance:** Constant time $O(1)$ for basic operations.

B. LinkedHashMap

- **Ordering:** Maintains the **insertion order** of keys.

C. TreeMap

- **Ordering:** Sorted according to the natural ordering of its keys.
- **Nulls:** Does not allow null keys